

## Test Case Classification Based on Groups

- **Login Test Case** → belongs to **Sanity** and **Master** groups.

```
Run ALL
public class TC_2_LoginTest extends BaseClass {

    @Test(groups={"Sanity","Master"})
    Run | Debug
    public void verif_Login() {
```

- **Account Registration Test Case** → belongs to **Regression** and **Master** groups.

```
Run ALL
public class TC_1_AccountRegistrationTest extends BaseClass {

    @Test(groups= {"Regression","Master"})
    Run | Debug
    public void verifyAccountRegistration() {
```

- **Login Data-Driven Test Case** → belongs to the **DataDriven** group.

```
Run ALL
public class TC_3_LoginDDT extends BaseClass {

    @Test(dataProvider="LoginData", dataProviderClass=DataProviders.class,groups={"DataDriven"})
    Run | Debug
    public void verify_loginDDT(String email, String password, String expect) {
```

BaseClass Group Execution Setup: Include all relevant groups in the TestNG **@BeforeClass** and **@AfterClass** annotations to ensure that the setup and teardown methods in the BaseClass are executed for all test groups.

```
@BeforeClass(groups={"Master","Sanity","Regression","DataDriven"})
@Parameters({"os", "browser"})
public void setup(String os, String br) throws IOException {
```

```
@AfterClass(groups={"Master","Sanity","Regression","DataDriven"})
public void teardown() {
```

Notes:

- **Master group** represents the **full suite**, meaning all tagged test cases will be executed.
- **DataDriven tests** are executed as a **separate group**, independent from other groups.

Grouping XML: Create a separate TestNG XML file named **grouping.xml** to define which groups to execute.

## Add Extent Reports to the Project

Create a **utility or listener** class dedicated to report configuration.

1. The **basic listener methods** commonly used are:
  - **onStart()** – initializes the Extent Report.
  - **onTestSuccess()** – logs passed test cases.
2. **Additional listener methods** include:
  - **onTestFailure()** – logs failed test cases and screenshots.

- `onTestSkipped()` – logs skipped tests.
- `onFinish()` – flushes and finalizes the report.

### Explanation of `onStart()` Method (Extent Reports + TestNG Listener):

#### 1. Creating a Date–Time Timestamp:

##### *First Method:*

- Create an object of the predefined `SimpleDateFormat` class to specify the desired pattern for generating the date and time.
- Create an instance of the predefined `Date` class to capture the **current system date and time**.
- Use the `format()` method of `SimpleDateFormat` and pass the `Date` object as a parameter: The method returns the formatted **timestamp as a String**.

```
SimpleDateFormat dateFormat = new SimpleDateFormat("yyyy.MM.dd.HH.mm.ss");
Date date = new Date();
String timeStamp = dateFormat.format(date);
```

##### *Second Method:*

Generate the current timestamp in a single step by creating a `SimpleDateFormat` instance, defining the desired pattern, and formatting a new `Date` object.

```
String timeStamp = new SimpleDateFormat("yyyy.MM.dd.HH.mm.ss").format(new Date());
```

#### 2. Generate Unique Report File Name Using Timestamp:

Append the generated timestamp and the `.html` extension to the report name to create a unique report file for each execution.

```
repName = "Test-Report" + timeStamp + ".html";
```

#### 3. Initialize ExtentSparkReporter with Report Path:

- Each new report file will be created inside the `reports` folder with the specified file name.
- The `ExtentSparkReporter` is initialized with the full path to the report.

```
sparkReporter = new ExtentSparkReporter(".\\reports\\" + repName);
```

#### 4. Attach Reporter and Set Project-Specific System Information:

- Attach the `ExtentSparkReporter` instance to the `ExtentReports` object to generate the HTML test report.

```
extent = new ExtentReports();
```

- The values for application name, module, sub-module, and environment are hardcoded using `setSystemInfo()` because they are constant for the project and describe the context of the test execution.

```

extent.attachReporter(sparkReporter);
extent.setSystemInfo("Application", "OpenCart");
extent.setSystemInfo("Module", "Admin");
extent.setSystemInfo("Sub Module", "Customers");
extent.setSystemInfo("User Name", System.getProperty("user.name"));
extent.setSystemInfo("Environment", "QA");

```

- `System.getProperty("user.name")`: Returns the name of the user who is running the test cases on the system.

#### 5. Explanation: `onStart(ITestContext testContext)`

```

public void onStart(ITestContext testContext) {

```

- The `onStart()` method is part of the **TestNG** `ITestListener` interface and is executed **before any test methods in the current <test> tag** start running.
- The parameter `testContext` contains all the **metadata and details** about the current test execution, including:
  - The **name of the test**
  - The **test methods will be executed**
  - Any **parameters** defined in the TestNG XML file
- The `testContext` variable can be used to store and share global data for reporting and logging during test execution.

#### Global Data (TestNG Context) - Notes:

**Global data** refers to information that can be accessed and used by multiple parts of the test execution. It is:

- **Shared data.**
- Available to **all test methods, listeners, and classes** within the same `<test>` scope.
- **Visible and reusable** throughout the test run.
- Used to **avoid recreating the same value** in every method.

✓ **In short:**

**Global data = shared, reusable information across the test execution scope.**

#### Examples of Global Data in Automation:

- Browser name (chrome, edge)
- Environment (QA, UAT, Prod)
- Report objects (ExtentReports instance)
- Login tokens / session IDs
- Generated test data (like dynamic email)
- Start time / execution ID

## TestNG execution flow - Notes:

```
<suite name="Suite">
  <test thread-count="5" name="Test">

    <parameter name="os" value="windows"/>
    <parameter name="browser" value="chrome"/>

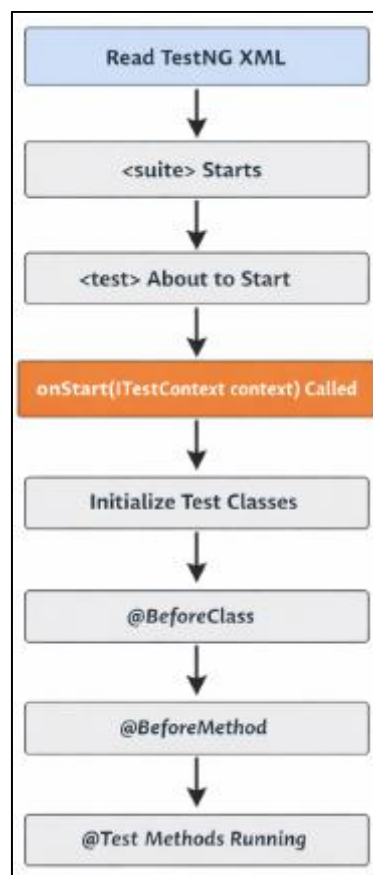
    <classes>
      <class name="testCases.TC_1_AccountRegistrationTest"/>
      <class name="testCases.TC_2_LoginTest"/>
      <class name="testCases.TC_3_LoginDDT"/>
    </classes>
  </test> <!-- Test -->
</suite> <!-- Suite -->
```

- 1) When TestNG reaches `<test name="Test">`
- 2) It starts running all classes and test methods inside it.
- 3) At this moment, the `onStart(ITestContext context)` listener method is triggered.

### ✓ **Exact Timing:**

It is triggered when the `<test>` tag begins execution, but before the first test class or test method is executed.

### TestNG execution flow diagram



- Retrieves **all the groups that are specified in the `<include>` section** of the TestNG XML file and stores them in a list.

```
List<String> includedGroups = testContext.getCurrentXmlTest().getIncludedGroups();
```

- Adds the **executed TestNG groups** information to the Extent Report system info section.

```
if (!includedGroups.isEmpty())  
    extent.setSystemInfo("Groups", includedGroups.toString());
```

- `getCurrentXmlTest()` → Returns the current `<test>` tag configuration from the XML file.
- `getIncludedGroups()` → Fetches the list of group names defined under `<groups><run><include>`.

```
<groups>  
  <run>  
    <!-- <include name="Master"/>  
    <include name="Sanity"/>  
    <include name="Regression"/> -->  
    <include name="DataDriven"/>  
    <!-- <exclude name=""/> -->  
  </run>  
</groups>
```

- `testContext` → Provides runtime information about the current `<test>` execution.
- `includedGroups.toString()` → converts the non-empty `List<String>` into a readable string format like: [Master, Regression]

- Reading Parameters from TestNG XML

```
String os = testContext.getCurrentXmlTest().getParameter("os");  
extent.setSystemInfo("os", os);  
  
String browser = testContext.getCurrentXmlTest().getParameter("browser");  
extent.setSystemInfo("browser", browser);
```

- `getParameter("os")` → Fetches the value of the parameter "os" defined in the XML.
- `getParameter("browser")` → Fetches the value of the parameter "browser" defined in the XML.

```
<test name="chrometest">  
  <parameter name="os" value="windows"/>  
  <parameter name="browser" value="chrome"/>  
  <classes>  
    <class name="testCases.TC_1_AccountRegistrationTest"/>  
  </classes>  
</test> <!-- Test -->
```

### Explanation of `onTestSuccess()` Method (Extent Reports + TestNG Listener):

```
public void onTestSuccess(ITestResult result) {
```

- When the `onTestSuccess(ITestResult result)` method is triggered, the result **object contains all execution details of the test method that just passed**.
- From the `ITestResult` object, we can extract information such as:
  - Test class name
  - Test method name
  - Execution status
  - Start and end time
  - Parameters

```
test = extent.createTest(result.getTestClass().getName());
```

- `result.getTestClass().getName()` → Retrieves the fully qualified class name of the executed test.

*Notes:* `getName()` → Full path (full package + class name)

*Example of code to get the executed test class simple name:*

```
test = extent.createTest(result.getTestClass().getRealClass().getSimpleName());
```

`getRealClass()` → returns the actual Java class.

`getSimpleName()` → Class name only (simple class name without package).

- `extent.createTest(...)` → Creates a new test entry in the Extent Report using that class name.

```
test.assignCategory(result.getMethod().getGroups());
```

- Access the **executed test method** using `result.getMethod()`.
- From that method, **extract the groups assigned** to it using `getGroups()`.
- **Assign these groups as categories** to the Extent Report test entry using `assignCategory()`.

```
test.log(Status.PASS, result.getName() + "Passed");
```

- Logs a **PASS status** in the Extent Report.
- Displays the **executed test method name**.
- Adds a custom message "Passed" next to the method name.

### Explanation of `onTestFailure()` Method (Extent Reports + TestNG Listener):

Same as `onTestSuccess()` method, with some additional handling.

```
test.log(Status.INFO, result.getThrowable().getMessage());
```

- Prints the error message (exception details) in the report.
  - Retrieves the **exception or error** that caused the test failure using `result.getThrowable()`.
  - Extracts the **error message** using `getMessage()`.
  - Logs that message in the **Extent Report**.

```
try {  
    String imgPath = new BaseClass().captureScreen(result.getName());  
    test.addScreenCaptureFromPath(imgPath);  
} catch (IOException e1) {  
    e1.printStackTrace();  
}
```

- If a failure occurs, capture a screenshot using the `captureScreen()` method implemented in the `BaseClass` and attach it to the report.

```
String imgPath = new BaseClass().captureScreen(result.getName());
```

- The `captureScreen()` method is implemented in the `BaseClass`.
- Since it is **not static**, it **cannot be called** → directly you must create an **instance** of `BaseClass` to access it.
- The test method name (`result.getName()`) is passed as a **parameter** to generate a unique screenshot file.

```
test.addScreenCaptureFromPath(imgPath);
```

- The returned path (`imgPath`) is then used to **attach the screenshot** to the Extent Report using `addScreenCaptureFromPath()`.

*Note:* Use a `try-catch` block to handle exceptions when a screenshot is not available. If an exception occurs, use the **predefined** `printStackTrace()` **method** to display the exception details.

Explanation `captureScreen()` Method:

```
String timeStamp = new SimpleDateFormat("yyyyMMddhhmmss").format(new Date());
```

- Creates a **timestamp string** representing the current system date and time.
- Used to generate **unique file names** (screenshots, reports, logs).
- Prevents file overwrite during multiple test executions.

```
TakesScreenshot takesScreenshot = (TakesScreenshot) driver;
```

- Casts the `WebDriver` **instance** to the `TakesScreenshot` **interface**.
- Enables the driver to **capture screenshots** during test execution.

```
File sourceFile = takesScreenshot.getScreenshotAs(OutputType.FILE);
```

- `getScreenshotAs(OutputType.FILE)` → captures a screenshot of the current browser state as a **file object**.
- `sourceFile` → Stores the screenshot in a **temporary system location**.

```
String targetFilePath = System.getProperty("user.dir") + "\\screenshots\\" + tname + "_" + timeStamp + ".png";
```

- Creates a **unique file path** for storing the screenshot that prevents overwriting files by using a unique timestamp.  
→ Provides a **reference path** for attaching screenshots to Extent Reports.

```
File targetFile = new File(targetFilePath);
```

- Creates a `File` **object** pointing to the location where the screenshot will be **permanently saved** in the project's screenshots folder.

```
sourceFile.renameTo(targetFile);  
return targetFilePath;
```

- Moves or renames the **sourceFile** (temporary screenshot) to the **targetFile** location (permanently screenshot).
  - The **returned screenshot path** (**targetFilePath**) is used to **attach** the **screenshot** to the **Extent Report**, while the actual file (**targetFile**) is **permanently stored** in the **screenshots** folder for failure analysis.

#### Managing WebDriver Instance Conflicts:

- In the **BaseClass**, the **captureScreen()** method uses the **WebDriver instance** that was already initialized in the **BaseClass**.
  - However, in the **ExtentReportManager** class, **a new object** of **BaseClass** is created. This may lead to the creation of **a new WebDriver instance**.
- As a result, **two different WebDriver instances** could be created: one managed by **the test execution** in **BaseClass**, and another created **inside ExtentReportManager**.

**Problem:** Two different **WebDriver instances** may **cause conflicts**, and the test execution may not behave as expected.

**Solution:** To ensure **a single shared driver instance** across multiple objects → **Declare the WebDriver as static in the BaseClass**.

→ This ensures that **the same driver instance** is shared and referenced across all objects.

```
public class BaseClass {
    static public WebDriver driver;
```

#### Explanation of onFinish() Method (Extent Reports + TestNG Listener):

```
extent.flush();
```

- 1) Consolidate all **test logs** that include **statuses**, **screenshots**, and **system information**.
- 2) Write everything to the **HTML** report.
- 3) Finalize report generation **at the end of execution**.

```
String PathOfExtentReport = System.getProperty("user.dir") + "\\reports\\" + repName;
```

- **Constructs the full file path** of the **Extent Report** by **combining** the **project root directory**, the reports folder, and the report name, and stores it in a String variable.

```
File extentReport = new File(PathOfExtentReport);
```

- **Creates a File object** (**extentReport**) that represents the generated Extent Report file.

```
try {
    Desktop.getDesktop().browse(extentReport.toURI());
} catch (IOException e2) {
    e2.printStackTrace();
}
```

- **Desktop** is a predefined Java class.



- This code **automatically opens** the generated **Extent Report** in the **default web browser**.
- Use a **try-catch block** to **handle exceptions** when a report file is not available or cannot be opened.

### Register Extent Report Listener in TestNG:

- To enable Extent Reports through a **TestNG Listener**, add the **listener entry** in the **testng.xml** file:

```
<listeners>
<listener class-name="utilities.ExtentReportManager"/>
</listeners>
```

### HTML Report Details:

The HTML report provides detailed test results, group-wise categorization, and graphical execution summaries for better analysis and traceability.

- 1) **First Page:** Displays the list of executed test cases along with their status (Pass/Fail/Skip). For failed test cases, the captured screenshots are also attached and displayed.

The screenshot shows the first page of the Extent Report for the test case **testCases.TC\_2\_LoginTest**. It includes a header with the test name, a timestamp of 02.16.2026 6:45:59 PM, a duration of 00:00:00:296, and a test ID of #test-id=3. Below the header, there are two tabs: "Sanity" and "Master". A screenshot of the application under test is displayed. The main content area shows a table with columns: STATUS, TIMESTAMP, and DETAILS. The first row shows a "Fail" status at 6:45:59 PM with the detail "verif\_LoginFailed". The second row shows an "Info" status at 6:45:59 PM with the detail "Login Failed expected [true] but found [false]".

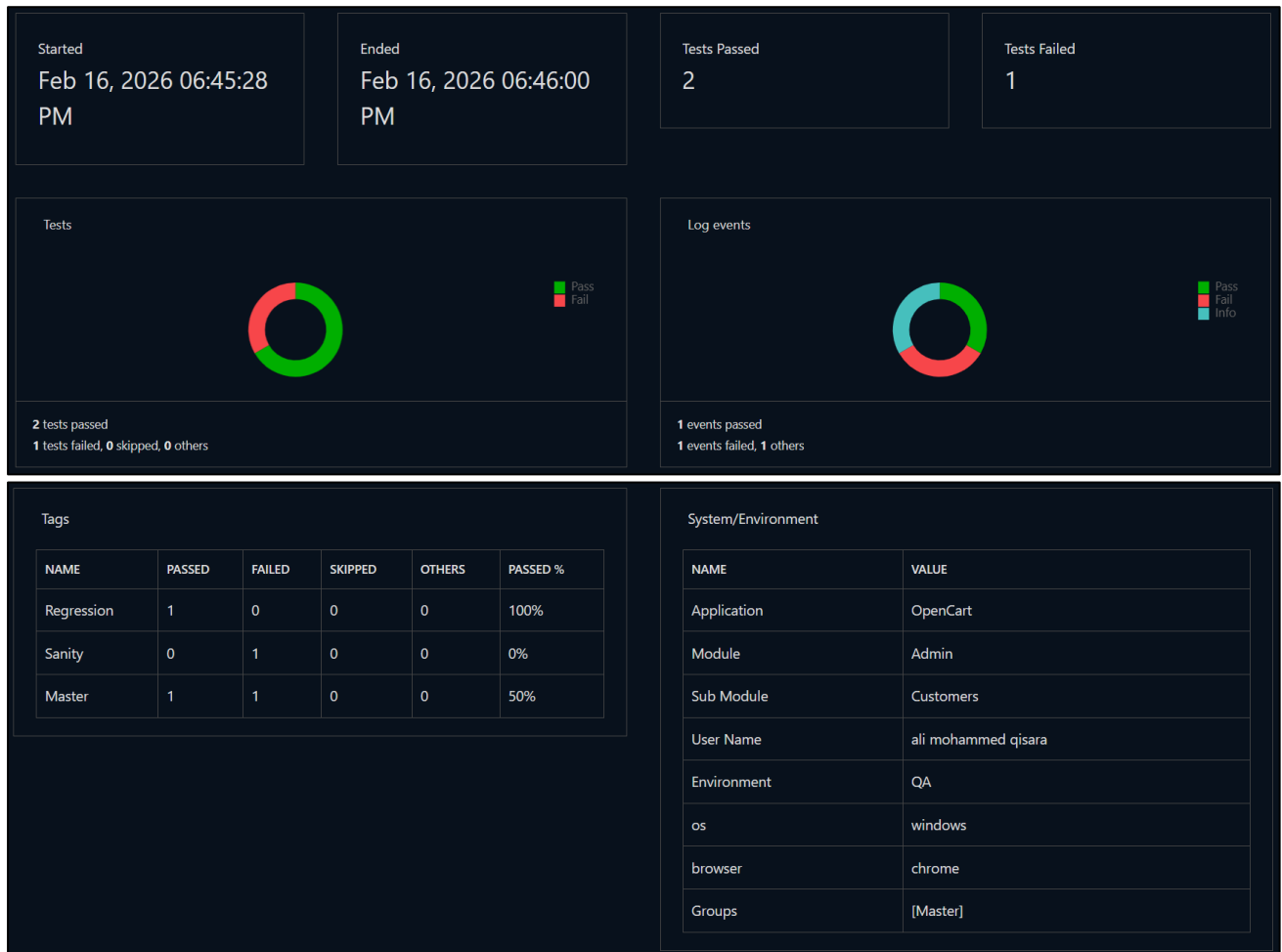
| STATUS | TIMESTAMP  | DETAILS                                        |
|--------|------------|------------------------------------------------|
| Fail   | 6:45:59 PM | verif_LoginFailed                              |
| Info   | 6:45:59 PM | Login Failed expected [true] but found [false] |

- 2) **Second Page:** Shows the results categorized group-wise (e.g., Smoke, Regression), allowing easy filtering and analysis.

The screenshot shows the second page of the Extent Report, displaying results categorized by group. On the left, there is a sidebar with categories: "Category" (3 tests), "Regression" (1 test), "Sanity" (1 test), and "Master" (2 tests). On the right, the "Master" category is expanded, showing a summary of 1 passed and 1 failed test. Below the summary, there is a table with columns: STATUS, TIMESTAMP, and TESTNAME. The first row shows a "Pass" status at 18:45:40 PM for the test "TC\_1\_AccountRegistrationTest". The second row shows a "Fail" status at 18:45:59 PM for the test "testCases.TC\_2\_LoginTest".

| STATUS | TIMESTAMP   | TESTNAME                     |
|--------|-------------|------------------------------|
| Pass   | 18:45:40 PM | TC_1_AccountRegistrationTest |
| Fail   | 18:45:59 PM | testCases.TC_2_LoginTest     |

- 3) **Third Page:** Presents the test execution results in graphical format, including charts and an overall execution summary.



## Run Failed Test Cases in TestNG

Scenario: Suppose we have **100 test cases**

- In the first execution: 90 test cases **pass** & 10 test cases **fail**
- In the second execution: We want to run **only the failed test cases**

Solution:

TestNG automatically generates a separate **XML file** containing only the failed test cases from the previous execution. This file is located in: test-output/testng-failed.xml

The screenshot shows the TestNG results page with two failed test cases:

- testCases.TC\_1\_AccountRegistrationTest**: 9:12:50 PM / 00:00:00:172 (Fail)
- testCases.TC\_2\_LoginTest**: 9:13:06 PM / 00:00:00:124 (Fail)

```

<classes>
  <class name="testCases.TC_1_AccountRegistrationTest">
    <methods>
      <include name="teardown"/>
      <include name="verifyAccountRegistration"/>
      <include name="setup"/>
    </methods>
  </class> <!-- testCases.TC_1_AccountRegistrationTest -->
  <class name="testCases.TC_2_LoginTest">
    <methods>
      <include name="teardown"/>
      <include name="verif_Login"/>
      <include name="setup"/>
    </methods>
  </class> <!-- testCases.TC_2_LoginTest -->
</classes>

```

#### Steps to Re-run Failed Test Cases:

- 1) Refresh the project.
- 2) Navigate to the `test-output` folder.
- 3) Open the `testng-failed.xml` file.
- 4) Run this XML file to execute only the failed test cases.

#### Notes:

- Even if the test cases pass in the next execution, the previous entries remain in the `testng-failed.xml` file until a new test execution updates it.
- When new test cases fail in a subsequent run, the `testng-failed.xml` file is regenerated and replaced with the latest failed test cases.